SE 1105 - PROGRAMMING AND PROBLEM SOLVING I

Scope of Variables, Function Calls

Asst. Prof. Dr. Kazım Erdoğdu

Faculty of Engineering
Department of Software Engineering
Yaşar University

Fall, 2024

Table of Contents

1. Scope of Variables
2. Function Calls
   - Memory View of a Function Call
   - Recursive Functions
3. Real Numbers

Table of Contents

1. Scope of Variables
2. Function Calls
   - Memory View of a Function Call
   - Recursive Functions
3. Real Numbers

Scope of Variables

- There are two types of variables in C regarding where they are defined in the program.
  - **Local Variables:** The variables defined in a block. They survive until the end of the block they are in.
  - **Global Variables:** The variables defined outside of any block. They survive until the termination of the program.
- In case of multiple variables with the same name, the one in the innermost block will always be accessed.

Scope of Variables

```
#include <stdio.h>
int a = 10;
void increment(int a, int b){
    a = a + b;
}
void change(){
    a = a * 2;
    printf("a: %d\n",a);
}
void main(){
    printf("a: %d\n",a);
    int a = 7;
    printf("a: %d\n",a);
    increment(a,5);
    printf("a: %d\n",a);
    change();
    printf("a: %d\n",a);
}
```

Scope of Variables

- Write a function called "swap" that receives two integers as input and exchanges their values.
- In the main() function, define two integer numbers and print them on the screen.
- Pass these two integers to the swap function. Inside the swap function, print the exchanged values of these integers on the screen.
- After the function call of swap, print the same integers on the screen?
- Does the swap function work as you expected? Why?

Table of Contents

1. Scope of Variables
2. Function Calls
   - Memory View of a Function Call
   - Recursive Functions
3. Real Numbers

Memory View of a Function Call

```c
#include <stdio.h>
void bar()
{
}
void foo()
{
    bar();
}
void main()
{
    foo();
}
```

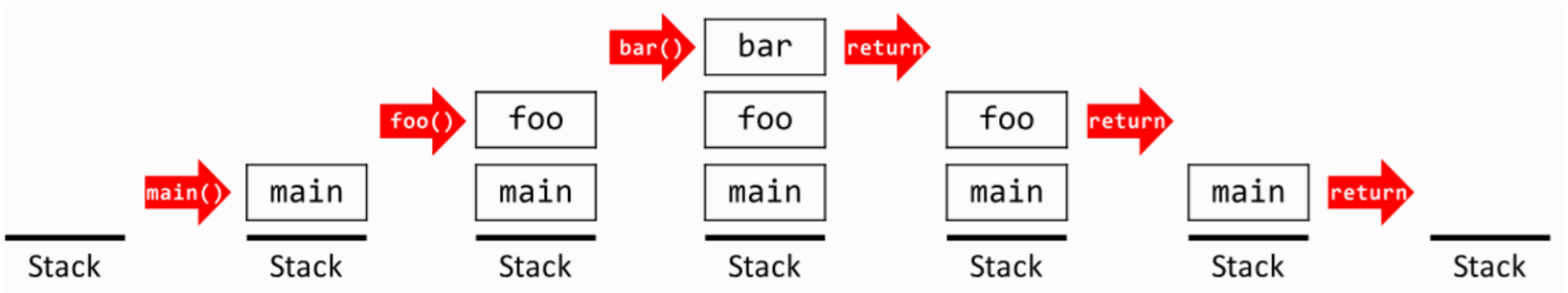

Figure: https://umich.sampleacademy.org/archive/eecs280/Section02

Memory View of a Function Call

- What does the memory view of the program on the right look like?
- What about the swap function example?

```c
#include <stdio.h>
int a = 10;
void increment(int a, int b){
    a = a + b;
}
void change(){
    a = a * 2;
    printf("a: %d\n",a);
}
void main(){
    printf("a: %d\n",a);
    int a = 7;
    printf("a: %d\n",a);
    increment(a,5);
    printf("a: %d\n",a);
    change();
    printf("a: %d\n",a);
}
```

Recursive Functions

- In programming, a recursive function is one that calls itself directly or indirectly through another function. (Deitel and Deitel, 2022).
- In fact, recursion is an advanced and complex topic discussed in detail in upper-level computer science courses.
- In this course, we will just do the introduction to it.

Recursive Functions

- A recursive function consists of two parts:
  - Base cases
  - Recursion steps

Recursive Factorial

```
int factorial(int n)
{
    if (n == 0){
        return 1;
    } else {
        return n * factorial(n-1);
    }
}
```

- How does the memory view look like when factorial(3) is called?

Recursive Functions

Recursive Fibonacci

```
int fibonacci(int n)
{
    if (n == 0 || n == 1){
        return 1;
    } else {
        return fibonacci(n-1) + fibonacci(n-2);
    }
}
```

- How does the memory view look like when fibonacci(5) is called?

Notes on Recursive Functions

- Constructing recursive functions requires effort and practice.
- They are often used in traversing graph and tree data structures, certain search algorithms (e.g. quick sort, merge sort), and mathematical formulas easily constructed recursively.
- The time and/or space complexities of the recursive functions are generally worse than their iterative counterparts (e.g. iterative fibonacci vs. recursive fibonacci).

Exercise

1. Write a function that receives positive integers m and n, and calculates $m * n$ without using $*$ and loops.

2. Write a function that receives positive integers m and n, and calculates m^n without using math.h library function and loops.

Table of Contents

1. Scope of Variables
2. Function Calls
   - Memory View of a Function Call
   - Recursive Functions
3. Real Numbers

Real Numbers

- The real numbers are represented with **float** and **double** data types in C.
- We will be using **float** mostly.

Type	Storage Size	Positive Value Range	Format Specifier
float	4 bytes	3.4×10^{-38} to 3.4×10^{38}	%f
double	8 bytes	1.7×10^{-308} to 1.7×10^{308}	%lf

Table: `https://www.geeksforgeeks.org/c-float-and-double/`

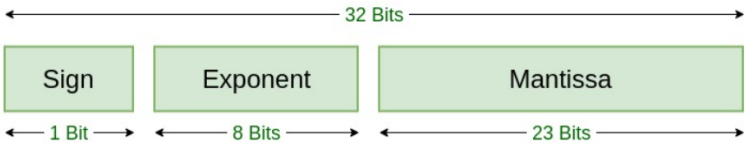

Figure: float precision

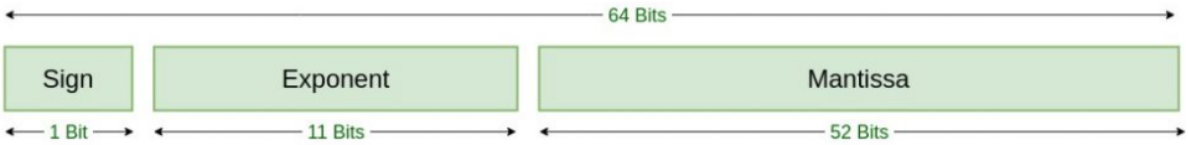

Figure: double precision

Exercise

1. Write a function that receives the height (in meters) and weight (in kilograms) of a person as float numbers and returns this person's body mass index (BMI).

$$BMI = \frac{weight}{height^2}$$

2. Write a function that receives the radius of a sphere and returns the volume of that sphere.

$$V = \frac{4}{3} * \pi * r^3$$

References

Deitel, P. and Deitel, H. (2022). *C How to Program*. Pearson, 9th edition.